



Universidad Politécnica
de Madrid



**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado en Ingeniería Informática

Trabajo Fin de Grado

**Diseño y Desarrollo de una Aplicación
de Facturación Automática con
Generación de Facturas y Cálculo de
Impuestos**

Autor: Tao Huang

Tutor(a): Nazario Félix González

Madrid, Enero 2026

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en Ingeniería Informática

Título: Diseño y Desarrollo de una Aplicación de Facturación Automática con
 Generación de Facturas y Cálculo de Impuestos

Enero 2026

Autor: Tao Huang

Tutor:

Nazario Félix González

Departamento de Arquitectura y Tecnología de sistemas Informáticos

ETSI Informáticos

Universidad Politécnica de Madrid

Resumen

Este Trabajo Fin de Grado presenta el diseño y desarrollo de una aplicación de facturación automática orientada a pequeñas y medianas empresas (PYMES) y trabajadores autónomos, con el objetivo de mejorar la eficiencia, fiabilidad y control de los procesos administrativos asociados a la emisión de facturas.

El sistema desarrollado, denominado SIF-PYME, permite la gestión integral de empresas, clientes, productos y facturas, incorporando el cálculo automático de impuestos conforme a la normativa fiscal vigente en España, incluyendo IVA, retenciones y recargos aplicables. Asimismo, la aplicación ofrece la generación y exportación de facturas en formatos digitales estándar como PDF y Excel, facilitando su almacenamiento y distribución.

El proyecto se ha abordado siguiendo una metodología propia de la ingeniería del software, que incluye el análisis de requisitos funcionales y no funcionales, el diseño de una arquitectura basada en el patrón Modelo-Vista-Controlador (MVC), el modelado de una base de datos relacional y la implementación del sistema utilizando tecnologías Java. Se ha prestado especial atención a la aplicación de buenas prácticas de desarrollo, la separación de responsabilidades y la mantenibilidad del código.

La validación del sistema se ha realizado mediante pruebas en un entorno real de una PYME, lo que ha permitido comprobar su correcto funcionamiento, estabilidad y adecuación a las necesidades del usuario final. Los resultados obtenidos confirman que la aplicación cumple los objetivos planteados y constituye una solución viable y práctica para la automatización de la facturación en entornos empresariales de pequeño y mediano tamaño.

Abstract

This Final Degree Project presents the design and development of an automatic invoicing application aimed at small and medium-sized enterprises (SMEs) and self-employed professionals, with the purpose of improving the efficiency, reliability, and accuracy of administrative invoicing processes.

The developed system, named SIF-PYME, provides comprehensive management of companies, clients, products, and invoices, incorporating automatic tax calculations in accordance with the current Spanish tax regulations, including VAT, withholdings, and applicable surcharges. In addition, the application supports the generation and export of invoices in standard digital formats such as PDF and Excel, facilitating document management and distribution.

The project has been carried out following a software engineering approach, including requirements analysis, architectural design based on the Model-View-Controller (MVC) pattern, relational database modelling, and system implementation using Java technologies. Particular emphasis has been placed on the application of good development practices, clear separation of responsibilities, and code maintainability.

System validation was performed through testing in a real SME environment, allowing the verification of its stability, usability, and suitability for real-world usage scenarios. The results demonstrate that the developed application meets the defined objectives and represents a practical and effective solution for automated invoicing in small business environments.

Tabla de contenidos

1	Introducción	1
1.1	Objetivos del TFG	1
2	Desarrollo	2
2.1	Requisitos del sistema	2
2.1.1	Requisitos funcionales (RF)	2
2.1.2	Requisitos no funcionales (RNF)	3
2.2	Restricciones del sistema (R)	3
2.3	Diseño de arquitectura	4
2.4	Diseño de base de datos	5
2.4.1	Diagrama entidad-relación	5
2.4.2	Descripción detallada de las entidades	5
2.4.2.1	Empresa	5
2.4.2.2	Cliente	5
2.4.2.3	Producto	6
2.4.2.4	Linea_factura	7
2.4.2.5	Factura	8
2.5	Herramientas de desarrollo	9
2.5.1	Justificación de decisiones técnicas	9
2.5.1.1	Elección de una aplicación de escritorio frente a una web	9
2.5.1.2	Elección de H2 frente a MySQL o PostgreSQL	10
2.5.1.3	Elección de Maven frente a Gradle	10
2.5.1.4	Elección de Swing frente a JavaFX u otras alternativas	10
2.6	Implementación y Buenas Prácticas de Desarrollo	11
2.6.1	Aplicación del principio DRY (Don't Repeat Yourself) [4]	11
2.6.2	Centralización de la apariencia visual: UIHelper y UITheme	11
2.6.3	Cumplimiento del Principio de Responsabilidad Única (Single Responsibility Principle) [5]	12
2.7	Testing	12
3	Resultados y conclusiones	13
3.1	Resultados obtenidos	13
3.2	Conclusiones	13
4	Análisis de Impacto	15
5	Bibliografía	16

1 Introducción

La gestión de la facturación constituye una tarea fundamental en el funcionamiento diario de pequeñas y medianas empresas (PYMES) y trabajadores autónomos. Sin embargo, en muchos casos estos procesos se realizan de forma manual o mediante herramientas poco adaptadas a las necesidades reales del negocio, lo que incrementa el riesgo de errores humanos, dificulta el cumplimiento de las obligaciones fiscales y reduce la eficiencia administrativa. En este contexto, el desarrollo de soluciones informáticas que automaticen y simplifiquen la generación de facturas resulta especialmente relevante.

El presente Trabajo Fin de Grado se centra en el diseño de una aplicación de facturación automática, orientada a PYMES y autónomos, que permita generar facturas a partir de los datos introducidos por el usuario de manera sencilla, fiable y conforme a la normativa fiscal vigente. La aplicación integra funcionalidades de gestión de clientes y productos, cálculo automático de impuestos y exportación de facturas en formato digital estándar, como PDF y Excel, con el objetivo de optimizar los procesos administrativos y facilitar su uso en entornos reales.

El desarrollo del proyecto se ha abordado siguiendo un enfoque propio de la ingeniería del software, que incluye el análisis de requisitos, el diseño de una arquitectura modular, la implementación del sistema y la validación mediante pruebas en escenarios de uso representativos. Asimismo, se ha prestado especial atención a la aplicación de buenas prácticas de diseño y a la creación de una interfaz de usuario intuitiva, que facilite la adopción del sistema por parte de usuarios sin conocimientos técnicos avanzados.

1.1 Objetivos del TFG

El objetivo principal de este Trabajo Fin de Grado es diseñar e implementar una aplicación de facturación automática que mejore la eficiencia y fiabilidad de los procesos administrativos en pequeñas empresas y autónomos. Para alcanzar este objetivo general, se han definido los siguientes objetivos específicos:

- Analizar los requisitos funcionales y no funcionales de una aplicación de facturación.
- Diseñar la arquitectura del sistema, la base de datos y el modelo de negocio.
- Implementar una aplicación con una interfaz de usuario intuitiva y accesible.
- Incorporar el cálculo automático de impuestos según diferentes tipos de IVA y retenciones.
- Permitir la generación de facturas en formatos digitales estándar, como PDF y Excel.
- Desarrollar módulos de gestión de clientes y productos.
- Documentar el proyecto conforme a las normas de la ingeniería del software.
- Validar y probar el sistema en escenarios de uso reales y representativos.

En conjunto, este trabajo busca proporcionar una solución práctica y aplicable que responda a una necesidad real del entorno empresarial, al mismo tiempo que demuestra la aplicación de los conocimientos adquiridos a lo largo del grado.

2 Desarrollo

2.1 Requisitos del sistema

En este apartado se identifican y clasifican los distintos requisitos del sistema propuesto, diferenciando entre requisitos funcionales (RF), requisitos no funcionales (RNF) y restricciones (R).

La clasificación y nomenclatura empleada se basa en las prácticas recomendadas por el estándar IEEE 830-1998, *Recommended Practice for Software Requirements Specifications*, que define las categorías de requisitos en inglés como *Functional Requirements* (FR), *Non-Functional Requirements* (NFR) y *Design Constraints o Design Restrictions* (C o R). En el presente trabajo se ha optado por adaptar dicha terminológica al español, empleando las siglas RF (Requisitos Funcionales), RNF (Requisitos No Funcionales) y R (Restricciones), manteniendo la equivalencia conceptual con la clasificación original del estándar. [1]

Asimismo, los requisitos no funcionales se han descrito considerando las características de calidad definidas por la norma ISO/IEC 25010:2011, *Systems and Software Quality Models*, que establece los atributos de calidad que debe cumplir un producto software. [2]

2.1.1 Requisitos funcionales (RF)

- RF1. El sistema debe permitir registrar, editar y eliminar datos de clientes (nombre, dirección, NIF, etc.).
- RF2. El sistema debe permitir registrar, editar y eliminar productos o servicios (nombre, precio, tipo de IVA).
- RF3. El usuario podrá introducir datos de factura, incluyendo número, fecha, método de pago y líneas de producto.
- RF4. El sistema debe calcular automáticamente los importes, incluyendo subtotal, impuestos (IVA, retenciones y recargos), y total final.
- RF5. El sistema debe permitir generar facturas en formato PDF y/o Excel.
- RF6. El usuario podrá guardar y consultar facturas anteriores desde la aplicación.
- RF7. El sistema debe permitir buscar y filtrar facturas por cliente, fecha o importe.
- RF8. El sistema debe incluir una interfaz gráfica intuitiva para la gestión de datos y generación de facturas.
- RF9. El sistema debe almacenar la información en una base de datos relacional.
- RF10. El sistema debe permitir exportar los datos de clientes o facturas para respaldo o migración.

2.1.2 Requisitos no funcionales (RNF)

- RNF1. Usabilidad:
La interfaz debe ser intuitiva y fácil de usar para usuarios no técnicos (autónomos o pequeñas empresas).
- RNF2. Rendimiento:
El sistema debe poder generar una factura en menos de 2 segundos tras la entrada de datos.
- RNF3. Fiabilidad:
Los cálculos de impuestos deben ser precisos y cumplir las normativas fiscales vigentes en el contexto de la normativa española vigente en el momento del desarrollo.
- RNF4. Mantenibilidad:
El código debe seguir buenas prácticas de programación y estar documentado para facilitar futuras mejoras.
- RNF5. Seguridad:
El acceso a la base de datos debe proteger la información sensible de clientes y facturas.
- RNF6. Escalabilidad:
La arquitectura debe permitir agregar nuevas funcionalidades como integración con contabilidad o envío automático de facturas por correo.

2.2 Restricciones del sistema (R)

- R1. El desarrollo se realizará utilizando un lenguaje orientado a objetos.
- R2. La base de datos será relacional.
- R3. La exportación de facturas se realizará en PDF y Excel, cumpliendo formatos estándar.
- R4. El sistema deberá ajustarse a la normativa fiscal vigente en España (IVA, retenciones IRPF).
- R5. El desarrollo total estará limitado a 297 horas de dedicación según la planificación del TFG.
- R6. Se debe emplear control de versiones con Git durante todo el desarrollo.

2.3 Diseño de arquitectura

Para el diseño de la arquitectura del sistema se ha adoptado el patrón de diseño Modelo-Vista-Controlador (MVC), ampliamente utilizado en el desarrollo de aplicaciones por su capacidad para separar las responsabilidades y mejorar la mantenibilidad y escalabilidad del software. [3]

El patrón MVC permite dividir la aplicación en tres capas principales, cada una con un rol bien definido:

- **MODEL (Modelo):** es la capa encargada de gestionar la lógica de negocio y el acceso a los datos. En el proyecto, esta capa se encuentra organizada en dos paquetes:
 - **entity:** contiene las clases que representan las entidades del dominio de la aplicación. Estas clases modelan los objetos que se corresponden con las tablas de la base de datos, encapsulando sus atributos y proporcionando métodos de acceso a los mismos.
 - **dao:** se encarga de realizar las operaciones de acceso a la base de datos. En él se implementan las consultas necesarias para insertar, actualizar, eliminar y recuperar información, aislando la lógica de persistencia del resto de la aplicación.
- **VIEW (Vista):** comprende las interfaces gráficas de usuario (GUI), organizadas por módulos funcionales, tales como cliente, producto y factura. Se mantiene una separación entre vistas de listado y formularios de edición, lo que favorece la claridad y la reutilización de componentes visuales.
- **CONTROLLER (Controlador):** Actúa como intermediario entre la vista y el modelo, gestionando la lógica de presentación, el flujo de datos y las interacciones del usuario. Los controladores reciben las acciones desde la vista, procesan la información y solicitan las operaciones necesarias al modelo o a los servicios correspondientes.

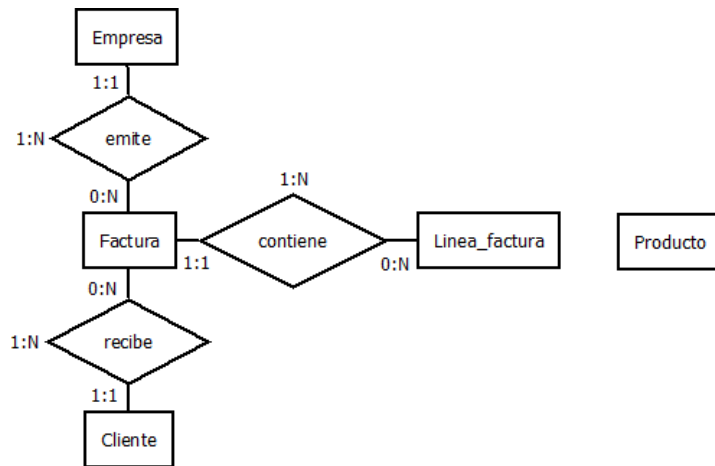
Además, la arquitectura se complementa con varios módulos auxiliares que refuerzan la organización del código y la reutilización de funcionalidades comunes:

- **service:** agrupa los servicios encargados de tareas específicas del sistema, como la generación de documentos y la realización de cálculos necesarios para la lógica del negocio.
- **util:** contiene clases de utilidad que proporciona servicios transversales a la aplicación como la gestión de la conexión con la base de datos y la navegación entre pantallas.

Esta estructura modular facilita la extensibilidad del sistema y contribuye a una mejor separación de responsabilidades, garantizando un mantenimiento más sencillo y una evolución futura más controlada del software.

2.4 Diseño de base de datos

2.4.1 Diagrama entidad-relación



2.4.2 Descripción detallada de las entidades

2.4.2.1 Empresa

Campo	Tipo	Descripción	Restricciones
id_empresa	INT	Identificador único	PK, AUTO_INCREMENT
razon_social	VARCHAR(255)	Razón social de la empresa	NOT NULL
nif	VARCHAR(20)	NIF o CIF de la empresa	UNIQUE, NOT NULL
direccion	VARCHAR(255)	Dirección fiscal completa	NULL
telefono	VARCHAR(20)	Teléfono de contacto	NULL
email	VARCHAR(100)	Correo electrónico de la empresa	NULL
por_defecto	BOOLEAN	Indica si es la empresa seleccionada por defecto	DEFAULT FALSE

2.4.2.2 Cliente

Campo	Tipo	Descripción	Restricciones
id_cliente	INT	Identificador único	PK, AUTO_INCREMENT
nombre_fiscal	VARCHAR(255)	Razón social del cliente	NOT NULL
nif	VARCHAR(20)	NIF o CIF del cliente	UNIQUE, NOT NULL
direccion	VARCHAR(255)	Dirección fiscal completa	NULL
telefono	VARCHAR(20)	Teléfono de contacto	NULL
email	VARCHAR(100)	Correo electrónico del cliente	NULL

2.4.2.3 Producto

Campo	Tipo	Descripción	Restricciones
id_producto	INT	Identificador único del producto	PK, AUTO_INCREMENT
tipo_iva	DECIMAL(5,2)	Porcentaje de IVA aplicado al producto	DEFAULT 21.00
codigo	VARCHAR(50)	Código interno del producto	UNIQUE
nombre	VARCHAR(255)	Nombre descriptivo del producto	NOT NULL
precio	DECIMAL(10,2)	Precio final del producto (con impuestos)	>= 0 o NULL
precio_base	DECIMAL(10,2)	Precio base sin impuestos	>= 0 o NULL
tipo_retencion	DECIMAL(5,2)	% de retención (IRPF)	DEFAULT 0
recargo_equivalencia	DECIMAL(5,2)	% de recargo de equivalencia	DEFAULT 0

Restricciones adicionales:

- Al menos uno de los campos precio o precio_base debe tener valor.
- Los valores monetarios no pueden ser negativos.
- El IVA, la retención y el recargo deben estar entre 0 y 100.

2.4.2.4 Línea factura

Campo	Tipo	Descripción	Restricciones
id_linea	INT	Identificador único de la línea	PK, AUTO_INCREMENT
id_factura	VARCHAR(20)	Factura a la que pertenece la línea	FK -> FACTURA, NOT NULL
nombre_producto	VARCHAR(255)	Descripción del producto o servicio	NOT NULL
cantidad	DECIMAL(10,2)	Cantidad facturada	>0
precio_base	DECIMAL(10,2)	Precio base por unidad	>=0
precio_unitario	DECIMAL(10,2)	Precio final por unidad	>=0
descuento	DECIMAL(5,2)	Descuento aplicado en porcentaje	DEFAULT 0, >=0, <=100
subtotal_linea	DECIMAL(10,2)	Subtotal antes de impuestos	>=0
porcentaje_iva	DECIMAL(5,2)	IVA aplicado a la línea	>=0
importe_iva	DECIMAL(10,2)	Importe de IVA de la línea	>=0
porcentaje_retencion	DECIMAL(5,2)	Retención aplicada	DEFAULT 0
importe_retencion	DECIMAL(10,2)	Importe de la retención	>=0
porcentaje_recargo	DECIMAL(5,2)	Recargo aplicado	DEFAULT 0
importe_recargo	DECIMAL(10,2)	Importe del recargo	>=0
total_linea	DECIMAL(10,2)	Total final de la línea	>=0
numero_linea	INT	Número de orden dentro de la factura	NOT NULL

2.4.2.4.1 Justificación de diseño de la entidad Línea_factura

La entidad Línea_factura se ha diseñado como un registro independiente que almacena todos los datos de cada producto o servicio incluido en una factura, en lugar de simplemente vincular la factura a un producto mediante una clave foránea. Esta decisión se basa en tres razones fundamentales:

1. Información contextual específica de la factura: Cada línea debe poder contener datos particulares de la transacción, como el orden de

presentación dentro de la factura, el descuento aplicado en ese momento, o modificaciones puntuales al precio, IVA... Esto permite que el usuario ajuste los valores para cada factura concreta, tal como se implementa en la interfaz de edición de líneas donde los campos son editables y los cálculos se realizan en tiempo real.

2. Inmutabilidad e integración histórica: Al guardar una copia completa de la descripción del producto, se garantiza que la factura emitida en el pasado permanezca inalterada, aunque el producto original cambie en el futuro (por ejemplo, si se actualiza su precio o tipo de IVA). Esto es crucial para cumplir con requisitos legales y auditorías, ya que las facturas son documentos legales que no deben modificarse a posteriori.
3. Desacoplamiento y flexibilidad: Este enfoque permite que las facturas incluyan productos que ya no existan en el catálogo, o servicios personalizados que no están registrados como productos. Además, facilita la portabilidad y el respaldo de los datos, ya que cada factura contiene toda la información necesaria para su interpretación sin depender de otras tablas.

En resumen, el patrón de “línea de factura independiente” asegura la consistencia temporal, la flexibilidad operativa y el cumplimiento normativo, que son requisitos críticos en un sistema de facturación profesional.

2.4.2.5 Factura

Campo	Tipo	Descripción	Restricciones
id_factura	VARCHAR(20)	Identificador único de la factura	PK
id_empresa	INT	Empresa emisora de la factura	FK -> EMPRESA, NOT NULL
id_cliente	INT	Cliente receptor de la factura	FK -> CLIENTE, NOT NULL
fecha_emision	DATE	Fecha de emisión de la factura	NOT NULL, DEFAULT CURRENT_DATE
metodo_pago	VARCHAR(50)	Método de pago utilizado	NOT NULL
subtotal	DECIMAL(10,2)	Importe total sin impuestos	>=0, DEFAULT 0
total_iva	DECIMAL(10,2)	Importe total del IVA	>=0, DEFAULT 0
total_retencion	DECIMAL(10,2)	Importe total de retenciones	>=0, DEFAULT 0
total_recargo	DECIMAL(10,2)	Importe total del recargo	>=0, DEFAULT 0
total	DECIMAL(10,2)	Importe total final de la factura	>=0, DEFAULT 0

2.5 Herramientas de desarrollo

Para el desarrollo del sistema, se ha seleccionado un conjunto de herramientas y tecnologías que garantizan la calidad, mantenibilidad y eficiencia del software.

Categoría	Herramientas	Propósito
Lenguaje	Java [7]	Lenguaje de programación principal
Base de Datos	H2 Database Engine [8]	Motor de base de datos embebida
Base de Datos	HikariCP [9]	Gestión eficiente de conexiones a la base de datos
Gestor de Dependencias	Apache Maven [10]	Construcción del proyecto y gestión de dependencias
Entorno de Desarrollo	Visual Studio Code [11]	Entorno de desarrollo integrado
Control de Versiones	Git [12]	Control de versiones distribuido
Control de Versiones	Github [13]	Alojamiento de código, revisión y CI/CD
Sistema de Registro	SLF4J [14]	API de logging y su implementación
Interfaz Gráfica	Java Swing [15]	Implementación de la interfaz gráfica de usuario

2.5.1 Justificación de decisiones técnicas

2.5.1.1 Elección de una aplicación de escritorio frente a una web

El enfoque de aplicación de escritorio se justifica por las necesidades y el perfil del público objetivo (pequeñas y medianas empresas). Las principales ventajas son:

- **Simplicidad de instalación y operación:** la aplicación puede distribuirse como un ejecutable/JAR auto-contenido, reduciendo la complejidad de instalación en entornos sin personal técnico dedicado.
- **Funcionamiento offline:** permite operar sin conexión a Internet, lo cual es crítico en entornos con conectividad intermitente.
- **Acceso directo a recursos locales:** facilita operaciones de E/S y acceso a dispositivos locales (impresoras fiscales, lectores de código, etc.).
- **Privacidad y seguridad de los datos:** los datos se almacenan localmente, lo que reduce el riesgo asociado a la exposición en red y facilita el cumplimiento de políticas de protección de datos cuando el cliente requiere almacenamiento local.

2.5.1.2 Elección de H2 frente a MySQL o PostgreSQL

H2 se ha seleccionado por su idoneidad para aplicaciones de escritorio y entornos con requisitos de instalación mínima:

- **Operación embebida:** elimina la necesidad de desplegar y administrar un servidor de base de datos independiente.
- **Portabilidad del almacenamiento:** la base de datos se materializa en un único fichero, lo que simplifica copias de seguridad y migración.
- **Adecuado para volúmenes moderados:** proporciona rendimiento suficiente para los usos previstos en PYMEs.
- **Compatibilidad SQL y modos de interoperabilidad:** facilita la adaptación posterior a otros motores si se requiere migración.

2.5.1.3 Elección de Maven frente a Gradle

Maven se ha preferido por razones prácticas y pedagógicas:

- **Declaratividad y convención:** su modelo basado en pom.xml es explícito y favorece la reproducibilidad de la construcción.
- **Madurez y abundante documentación:** facilita que nuevos colaboradores comprendan rápidamente la estructura del proyecto.
- **Integración con IDEs y herramientas de análisis estático:** soporte consolidado en VS Code y en herramientas de CI/CD.
- **Comportamiento predecible:** para proyectos de tamaño medio y con requisitos de compilación estables, Maven aporta menos sorpresas frente a configuraciones complejas.

2.5.1.4 Elección de Swing frente a JavaFX u otras alternativas

Swing se ha seleccionado como biblioteca para la implementación de la interfaz gráfica por motivos de madurez, compatibilidad y adecuación al alcance del proyecto:

- **Integración nativa con el JDK:** al estar incluida en la biblioteca estándar de Java, no requiere configuraciones adicionales ni dependencias externas, lo que simplifica la compilación y distribución del sistema.
- **Madurez y estabilidad comprobada:** Swing cuenta con un largo historial de uso en aplicaciones empresariales, con una API estable y ampliamente documentada.
- **Compatibilidad multiplataforma:** su soporte nativo dentro del ecosistema Java garantiza un comportamiento consistente en distintos sistemas operativos.
- **Dependencias reducidas:** a diferencia de frameworks como JavaFX o SWT, no requiere bibliotecas externas ni componentes nativos, lo que favorece la portabilidad y reduce el mantenimiento.
- **Adecuación al alcance del proyecto:** dado que el sistema prioriza la funcionalidad y la estabilidad sobre la apariencia visual avanzada, Swing satisface plenamente los requisitos sin añadir complejidad innecesaria.

2.6 Implementación y Buenas Prácticas de Desarrollo

La implementación del proyecto puede consultarse en el siguiente repositorio de GitHub: <https://github.com/Tao-Huang-TFG/TFG-SIF-PYME>. A lo largo del desarrollo se han aplicado diversas buenas prácticas de programación con el objetivo de mejorar la calidad, mantenibilidad y consistencia del código.

2.6.1 Aplicación del principio DRY (Don't Repeat Yourself) [4]

Para evitar la duplicación de código y facilitar su reutilización, se han implementado las clases base `BaseFormView` y `BaseListView`. Estas clases encapsulan la funcionalidad común a todas las vistas de formularios y listados respectivamente, tales como:

- Configuración de layouts y estilos visuales.
- Gestión de botones estándar (guardar, cancelar, editar, eliminar).
- Lógica de navegación entre pantallas mediante `CardLayout`.
- Validación básica de campos y presentación de mensajes de error.

De este modo, las vistas concretas (por ejemplo, `ClienteFormView` y `FacturasView`) heredan de estas clases base y solo deben implementar los métodos abstractos correspondientes a su lógica específica, reduciendo significativamente la redundancia y mejorando la cohesión del código.

2.6.2 Centralización de la apariencia visual: `UIHelper` y `UITheme`

Con el fin de garantizar una experiencia de usuario uniforme en toda la aplicación, se ha desarrollado un sistema centralizado de estilos mediante dos clases:

- `UITheme`: Define de manera unificada los colores, fuentes e iconos utilizados en los diferentes módulos (clientes, empresas, productos, facturas). Por ejemplo, el color primario para el módulo de facturas (`COLOR_FACTURAS`) o los iconos Unicode utilizados en los botones (`ICONO_GUARDAR`, `ICONO_ELIMINAR`).
- `UIHelper`: Proporciona métodos de factoría para la creación de componentes Swing estilizados (botones, campos de texto, áreas de texto, combobox, paneles de selección), asegurando que todos sigan el mismo estilo definido en `UITheme`.

Esta aproximación no solo asegura la consistencia visual, sino que también simplifica la tarea de realizar cambios globales en la interfaz, ya que las modificaciones se realizan en un único punto.

2.6.3 Cumplimiento del Principio de Responsabilidad Única (Single Responsibility Principle) [5]

La arquitectura MVC descrita en la sección 2.3 se ha aplicado rigurosamente para mantener una clara separación de responsabilidades. El modelo se limita a la gestión de datos y la lógica de negocio, la vista gestiona exclusivamente la presentación y la interacción con el usuario, y los controladores coordinan el flujo de información entre ambas capas.

Esta separación se refuerza mediante la creación de servicios especializados con funciones bien definidas como `CalculadoraFacturaService`, para los cálculos de facturación y servicios independientes para la generación de documentos en PDF y Excel. Asimismo, las funcionalidades transversales, como la conexión a la base de datos o la navegación entre vistas se encapsulan en clases específicas, evitando mezclar responsabilidades en componentes no relacionados.

Gracias a esta organización, cada clase mantiene una función concreta y bien delimitada, lo que facilita el mantenimiento del código, reduce el acoplamiento entre componentes y permite realizar modificaciones de forma localizada sin afectar al resto del sistema.

2.7 Testing

El proceso de pruebas del sistema se ha llevado a cabo mediante un enfoque de Quality Assurance (QA)[6] en un entorno real, concretamente en una empresa PYME dedicada al comercio minorista de alimentación, con una plantilla aproximada de diez empleados, lo que ha permitido validar la aplicación en condiciones de uso reales. Este enfoque ha facilitado la detección temprana de errores y la incorporación de mejoras a partir de las peticiones directas del usuario final, alineando el desarrollo del sistema con sus necesidades reales.

La utilización de QA ha resultado especialmente relevante debido a la evolución progresiva de los modelos de datos durante el desarrollo del proyecto. A medida que se identificaban con mayor precisión las variables necesarias para cada tabla de la base de datos, las pruebas continuas permitían verificar la coherencia de los cambios y su correcto impacto en el funcionamiento del sistema.

Asimismo, este enfoque ha permitido realizar un testing más efectivo de las vistas y de la interacción con el usuario, evaluando tanto la funcionalidad como la usabilidad de la aplicación. De este modo, el proceso de QA ha contribuido a mejorar la estabilidad, fiabilidad y calidad global del sistema antes de su despliegue definitivo.

3 Resultados y conclusiones

3.1 Resultados obtenidos

El desarrollo del presente Trabajo Fin de Grado ha dado lugar a una aplicación de facturación automática plenamente funcional, denominada SIF-PYME, que cumple los objetivos definidos en las fases iniciales del proyecto.

La aplicación satisface la totalidad de los requisitos funcionales especificados en la Sección 2.1, lo que se verificó durante el proceso de validación del sistema. Los módulos de gestión de empresas, clientes, productos y facturas permiten realizar operaciones completas de creación, consulta, modificación y eliminación de datos, así como la generación automática de facturas ajustadas a la normativa fiscal vigente. Asimismo, el sistema permite la exportación de documentos en formatos digitales estándar como PDF y Excel.

La arquitectura basada en el patrón Modelo-Vista-Controlador, descrita en la Sección 2.3, ha demostrado ser adecuada para el alcance del proyecto, facilitando una clara separación de responsabilidades y contribuyendo a la mantenibilidad y extensibilidad del código. El diseño de la base de datos garantiza la integridad referencial y la conservación de la información histórica de las facturas emitidas.

La interfaz gráfica desarrollada con Java Swing proporciona todas las funcionalidades necesarias para la operación del sistema y fue validada en condiciones reales de uso, confirmando que resulta comprensible y operativa para usuarios sin conocimientos técnicos avanzados.

El sistema fue probado en un entorno real de una PYME, lo que permitió confirmar su estabilidad y correcto funcionamiento bajo condiciones habituales de uso. Durante estas pruebas se verificó el cumplimiento de los requisitos de rendimiento, constatando que la generación de facturas se realiza en un tiempo inferior a dos segundos.

3.2 Conclusiones

El desarrollo de SIF-PYME ha permitido alcanzar el objetivo principal del Trabajo Fin de Grado: diseñar e implementar una aplicación de facturación automática que mejore la eficiencia y fiabilidad de los procesos administrativos en pequeñas empresas y trabajadores autónomos.

Todos los objetivos específicos planteados al inicio del proyecto se han cumplido satisfactoriamente. Se llevó a cabo un análisis estructurado de requisitos, se diseñó una arquitectura modular y una base de datos adecuada al dominio del problema, se implementó una interfaz de usuario funcional y se incorporó el cálculo automático de impuestos conforme a la normativa vigente.

El diseño de las líneas de factura independientes, descrito en la Sección 2.4, se confirmó durante el desarrollo como una decisión clave para garantizar la inmutabilidad de la información y el cumplimiento de los requisitos legales asociados a la facturación, incluso a costa de una mayor redundancia de datos.

La aplicación sistemática de buenas prácticas de desarrollo y patrones de diseño contribuyó de forma decisiva a mantener un código organizado, reutilizable y fácil de mantener, facilitando la evolución del sistema a lo largo del proyecto.

Desde el punto de vista formativo, el desarrollo del TFG ha permitido consolidar conocimientos en diseño de bases de datos relacionales, programación orientada a objetos en Java, desarrollo de interfaces gráficas, gestión de dependencias y control de versiones, así como adquirir experiencia en la validación de un sistema software en un entorno real.

Como líneas de trabajo futuro, se identifican posibles ampliaciones del sistema, tales como la implementación de copias de seguridad automáticas, la integración con sistemas contables externos, el envío automatizado de facturas por correo electrónico o el soporte para entornos multiusuario. Estas extensiones, aunque fuera del alcance del presente trabajo, podrían incorporarse en versiones posteriores del sistema.

4 Análisis de Impacto

La implementación de SIF-PYME tiene el potencial de generar impactos positivos en distintos ámbitos, especialmente en el contexto de pequeñas y medianas empresas y trabajadores autónomos que requieren soluciones de facturación sencillas, fiables y accesibles.

Desde el punto de vista empresarial, la automatización del proceso de facturación permite reducir la carga administrativa asociada a tareas repetitivas y minimizar errores humanos en los cálculos fiscales. Esto contribuye a una mayor eficiencia operativa y facilita el cumplimiento de las obligaciones legales, mejorando la organización interna de los negocios que adoptan el sistema.

El diseño de la aplicación como solución de escritorio con base de datos embebida reduce significativamente las barreras de entrada tecnológicas. Al no requerir infraestructuras complejas ni conectividad permanente a internet, el sistema resulta especialmente adecuado para microempresas y autónomos con recursos técnicos y económicos limitados, favoreciendo la adopción de herramientas digitales de gestión.

En el ámbito social y organizacional, la utilización de una interfaz intuitiva y flujos de trabajo simplificados facilita la incorporación de usuarios con distintos niveles de alfabetización digital, contribuyendo a la digitalización progresiva de negocios tradicionales. Asimismo, al facilitar el cumplimiento de obligaciones fiscales y documentales, el sistema favorece prácticas administrativas más ordenadas y profesionales.

Desde una perspectiva medioambiental, la generación y almacenamiento de facturas en formato digital reduce la necesidad de impresión de documentos físicos, contribuyendo a una disminución del consumo de papel en los procesos administrativos habituales.

El proyecto presenta, además, una alineación indirecta con varios Objetivos de Desarrollo Sostenible de la Agenda 2030, en particular aquellos relacionados con el crecimiento económico inclusivo, la innovación tecnológica y el uso responsable de recursos, al promover la digitalización eficiente de procesos administrativos en pequeñas empresas.

En conjunto, el análisis realizado pone de manifiesto que SIF-PYME constituye una solución técnica viable y adecuada a su contexto de uso, con un impacto positivo razonable en términos de eficiencia, accesibilidad y sostenibilidad, coherente con los objetivos y el alcance de un Trabajo Fin de Grado en Ingeniería Informática.

5 Bibliografia

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)*, IEEE Computer Society, 1998.

[2] ISO/IEC, *ISO/IEC 25010:2011 – Systems and Software Quality Requirements and Evaluation (SQuaRE): System and Software Quality Models*, International Organization for Standardization, 2011.

[3] G. E. Krasner and S. T. Pope, “A cookbook for using the model-view controller user interface paradigm in Smalltalk-80,” *Journal of Object-Oriented Programming*, vol. 1, no. 3, pp. 26–49, 1988.

[4] A. Hunt and D. Thomas, *The Pragmatic Programmer: From Journeyman to Master*, ch. 2, “The Evils of Duplication,” Addison-Wesley Professional, 1999.

[5] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, ch. 2, Prentice Hall, 2002.

[6] IEEE, *IEEE Standard for Software Quality Assurance Processes (IEEE Std 730-2014)*, IEEE Computer Society, 2014.

[7] Oracle, “Java Platform, Standard Edition,” Oracle Corporation. [Online]. Available: <https://www.oracle.com/java/>. Accessed: Jan. 2026.

[8] H2 Database, “H2 Database Engine,” H2. [Online]. Available: <https://www.h2database.com/>. Accessed: Jan. 2026.

[9] B. Wooldridge, “HikariCP – A solid, high-performance JDBC connection pool,” GitHub repository. [Online]. Available: <https://github.com/brettwooldridge/HikariCP>. Accessed: Jan. 2026.

[10] Apache Software Foundation, “Apache Maven Project,” Apache. [Online]. Available: <https://maven.apache.org/>. Accessed: Jan. 2026.

[11] Microsoft, “Visual Studio Code,” Microsoft Corporation. [Online]. Available: <https://code.visualstudio.com/>. Accessed: Jan. 2026.

[12] S. Chacon and Others, “Git,” Software Freedom Conservancy. [Online]. Available: <https://git-scm.com/>. Accessed: Jan. 2026.

[13] GitHub, Inc., “GitHub,” GitHub. [Online]. Available: <https://github.com/>. Accessed: Jan. 2026.

[14] QOS.ch, “SLF4J – Simple Logging Facade for Java,” QOS.ch. [Online]. Available: <https://www.slf4j.org/>. Accessed: Jan. 2026.

[15] Oracle, “The Java Tutorials – Swing,” Oracle Corporation. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/>. Accessed: Jan. 2026.